Deep Reinforcement Learning

Value function approximation

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Content

Content

- Limitations of tabular methods
- Value estimation / prediction with function approximation
- Gradient-based prediction
- Batch learning

Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6	Brief introduction to deep learning	The basics for what comes next
7	Value function approximation	Value estimation with function approximation
8	Deep Q-learning	
9	Policy gradients	
10	Actor-critic algorithms	
11	Advanced algorithms (Part I): From policy gradient to PPO	
12	Advanced algorithms (Part II): From Q-learning to Soft Actor-Critic	
13	Exploration	
	Model-Based Control	
	Advanced Topics	

Chapter	Topic	Content

Lecture contents

Limitations of tabular methods

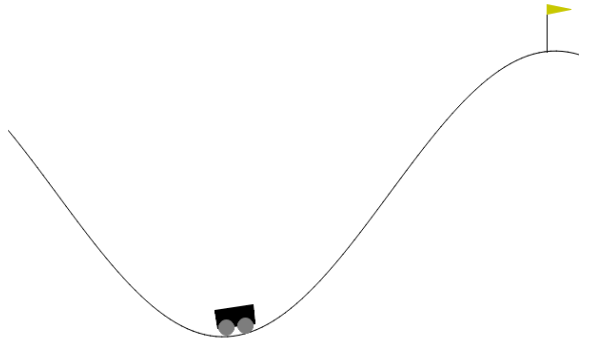
Limitations of tabular methods

We have essentially "solved" the RL problem in part one of our lecture! But there are quite a few limitations when it comes to real-world systems:

- The **curse of dimensionality**: For large dimensions $|\mathcal{S}|$ or $|\mathcal{A}|$ (even moderate ones), the calculations become prohibitively expensive and convergence is very slow.
- **Continuous state spaces**, i.e., infinitely many possible states

$$s \in \mathcal{S} \subseteq \mathbb{R}^n \quad / \quad |\mathcal{S}| = \infty. \qquad (\text{instead of } \mathcal{S} = \{s_1, \dots, s_{|\mathcal{S}|}\})$$

  - **Example**: The *mountain car*, with continous $\mathcal{S} = [x_{\text{min}}, x_{\text{max}}] \times [v_{\text{min}}, v_{\text{max}}]$ (minimal and maximal position x and velocity v), but finite $\mathcal{A} = \{-1, 0, 1\}$ (accelerate left, do nothing, accelerate right).

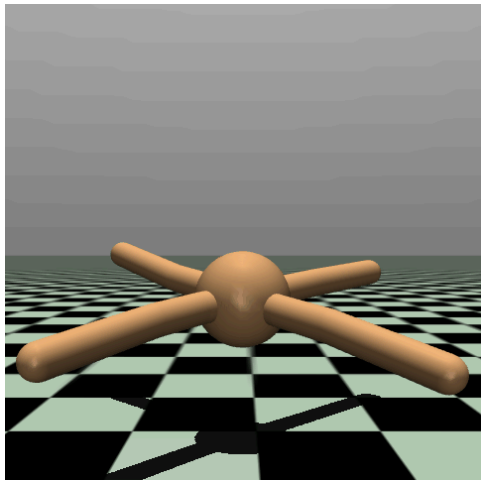


Mountain car [Source].

- **Continuous action spaces**, i.e., infinitely many possible actions*

$$a \in \mathcal{A} \subseteq \mathbb{R}^m \quad / \quad |\mathcal{A}| = \infty. \qquad (\text{instead of } \mathcal{A} = \{a_1, \dots, a_{|\mathcal{A}|}\})$$

  - **Example**: The *ant*, with 27 continous state variables and 8 continuous action variables.



* For now, however, we will stick to finite action spaces.

Ant [Source].

Value estimation / prediction with function approximation

Value estimation / prediction with function approximation

- From now on, we will introduce function approximators for the quantities of interest.
- For the value function, this means that we're looking for

$$V_\theta(s) \approx V^\pi(s).$$

  - $V_\theta(s)$ may be a *linear model*, i.e.,

$$V_\theta(s) = \sum_{k=1}^{d} \theta_k \psi_k(s) = \theta_1 \psi_1(s) + \ldots + \theta_d \psi_d(s),$$

  - but also a *nonlinear model* such as a deep neural network.
- The weight vector $\theta \in \mathbb{R}^d$ typically is of lower dimension d than the state space $\mathcal{S}$: $d \ll |\mathcal{S}|$.
  - Otherwise this exercise would be pointless!
  - This obviously holds for continuous state spaces, but also for very large yet finite-dimensional state spaces.

A note on the state space dimension
We use n to denote the number of state variables, but this does not mean that we need to have $n = \infty$ for continuous state spaces. A state space is finite if each state variable s_k is an element of a finite set, whereas it is continous, if at least one $s_1, \ldots, s_n$ is a continuous variable.

Global impact of experience
Updates due to new experience lead to updates in our model parameter θ. This means that *we modify our value estimate for many states*, in contrast to tabular methods where we update individual states only.

💡 We will use greek letters (e.g., θ or ϕ) to indicate the trainable parameters.

Prediction objective

- In the tabular case a specific prediction objective was not needed:
  - The learned value function could exactly match the true value.
  - The value estimate at each state was decoupled from other states.
- Due to global impact of experience, we need to define an accuracy metric on the entire state space: the **RL prediction goal**.

Definition: Value error (VE)
The RL prediction objective is defined as the mean squared value error

$$\overline{VE}(\theta) = \int_{\mathcal{S}} \mu(s) \Big(V^{\pi}(s) - V_{\theta}(s) \Big)^2 \, \mathrm{d}s.$$

Here, $\mu(s) \geq 0$ is the *state distribution*, with $\int_{\mathcal{S}} \mu(s) \, \mathrm{d}s = 1$.

Challenge:

- The true value $V^{\pi}(s)$ is unknown in most tasks!
- We need to find ways to estimate $V^{\pi}(s)$ from experience.

The on-policy prediction objective

- For prediction we will **focus entirely on the on-policy case**.
- Hence, $\mu(s)$ is the on-policy distribution under the current policy π.

On policy prediction objective
As $\overline{VE}(\theta)$ is the expected error, we can use *Monte Carlo sampling* to estimate it:

$$L(\theta) = \sum_{k=1}^{N} \left(V^{\pi}(s_k) - V_{\theta}(s_k)\right)^2 \approx \overline{VE}(\theta). \tag{1}$$

Goal: find the optimal value estimate V_{θ^*} by optimization:

$$\theta^* = \arg\min_{\theta} L(\theta).$$

Challenges:

1. The true value $V^{\pi}(s)$ is unknown in most tasks!
2. The function approximator $V_{\theta}(s)$ needs to be able to approximate $V^{\pi}(s)$.
3. Training:

- If $V_{\theta}(s)$ is a linear model: convex optimization problem.
  - The “easy” case: local optimum = global optimum, uniquely defined.
- If $V_{\theta}(s)$ is nonlinear model: nonlinear optimization problem.
  - The “hard” case: many local optima with no guarantee to locate the global one.
  - Training may diverge, advanced optimization techniques are very important!

💡 **Off-policy prediction**: *importance sampling* to transform the sampled value estimates from the behavior to the target policy.
⇒ Increases the prediction variance; in combination with generalization errors due to function approximation: large risk of diverging.

Gradient-based prediction

Incremental weight updates

- Now that we have an objective $L(\theta)$ we would like to optimize for (Eq. (1)), let's use **gradient descent** with learning rate α:

$$\theta^{(t+1)} = \theta^{(t)} - \frac{1}{2}\alpha\nabla L\left(\theta^{(t)}\right) = \theta^{(t)} - \frac{1}{2}\alpha\nabla_\theta\left(\sum_{k=1}^{N}\left(V^\pi(s_k) - V_\theta(s_k)\right)^2\right).$$

- What's the problem here?
  $\Rightarrow$ We often only have a *single sample*. (Or, instead, we would like to perform an update after a single sample.)

SGD-based parameter update
Given a new sample s_t and its (for now exact) value $V^\pi(s_t)$, we can update our current weight vector $\theta^{(t)}$ by using **stochastic gradient descent** (**SGD**) with:

$$\theta^{(t+1)} = \theta^{(t)} - \frac{1}{2}\alpha\nabla_\theta\left[\left(V^\pi(s_t) - V_\theta(s_t)\right)^2\right] = \theta^{(t)} + \alpha[V^\pi(s_t) - V_\theta(s_t)]\nabla_\theta V_\theta(s_t). \tag{2}$$

Question: Should we follow (2) to optimality?
$\Rightarrow$ No! We are optimizing for a single sample $\rightarrow$ **overfitting**.
$\Rightarrow$ Take a single, small step only, then collect new experience.

Bootstrapping the true value

Challenge 1. remains: We do not know $V^\pi(s_t)$ which we need for our prediction objective (1).

What can we do? $\Rightarrow$ Use an estimate for $V^\pi(s_t)$ that we can compute from experience!

Possible options for estimates:

Unbiased estimates of $V^\pi(s_t)$
Monte Carlo estimates for g_t, sample from entire trajectories.

Bootstrapped estimates of $V^\pi(s_t)$

- The Dynamic Programming target

$$V_\theta(s_t) \leftarrow \sum_{a \in \mathcal{A}} \pi\left(a|s_t\right) \sum_{s' \in \mathcal{S}} p\left(s'|s_t, a\right)\left[r + \gamma V_\theta(s')\right].$$

- TD targets (e.g., TD(0))

$$V_\theta(s_t) \leftarrow V_\theta(s_t) + \alpha\left[r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)\right].$$

💡 Inserting the bootstrapped versions into (2) does not yield a true gradient descent method, as we are ignoring the parametric dependency of the target on θ (Sutton and Barto 1998). More on this on the next slides.

Algorithm: Gradient Monte Carlo value estimation

Algorithm: Gradient Monte Carlo algorithm for estimating $V_\theta \approx V^\pi$
Input:

- The current policy π
- a differentiable, parameter-dependent function $V_\theta : \mathcal{S} \to \mathbb{R}$
- learning rate α

Initialize: Value function weights $\theta \in \mathbb{R}^d$ arbitrarily

for $k = 1, 2, \dots, K$ episodes:
Generate a sequence following π:

$$((s_0, a_0, r_0), (s_1, a_1, r_1), \dots, (s_{T_k-1}, a_{T_k-1}, r_{T_k-1}))$$

Calculate the every-visit returns g_t
for $t = 0, 1, \dots, T-1$:
$\theta \leftarrow \theta + \alpha[g_t - V_\theta(s_t)]\nabla_\theta V_\theta(s_t)$

- Direct transfer from the tabular case to function approximation.

Semi-gradient methods

- If bootstrapping is applied, the true target $V^\pi(s_t)$ is approximated by a target depending on the estimate $V_\theta(s_t)$.
- If $V_\theta(s_t)$ does not fit $V^\pi(s_t)$ (which it does not before convergence), then the update target becomes a *biased estimate*.
  - For example, in the TD(0) case, we get for (1):

$$L(\theta) = \sum_{t=1}^{T} \Big(r_t + V_\theta(s_{t+1}) - V_\theta(s_t)\Big)^2.$$

  - Taking the gradient of the single-sample version for $L(\theta)$ yields:

$$\begin{aligned} \theta \leftarrow\ & \theta - \frac{1}{2}\alpha\nabla_\theta \left[(r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t))^2\right] \\ & = \theta - \alpha[r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)]\ \nabla_\theta[\gamma V_\theta(s_{t+1}) - V_\theta(s_t)] \\ & \neq \theta + \alpha[r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)] \qquad\quad \nabla_\theta V_\theta(s_t) \end{aligned}$$

(Eq. (2)).

  - Application of Eq. (2) is still very common.
  - These appraoches are known as **semi-gradients** because they do not approximate the actual gradient.

TD(0) semi-gradient update of θ

$$\theta \leftarrow \theta + \alpha[r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)]\nabla_\theta V_\theta(s_t). \tag{3}$$

💡 Even though widely applied, there are no convergence guarantees for semi-gradient methods (Sutton and Barto 1998).

Algorithm: Semi-gradient TD(0) value estimation

Algorithm: Semi-gradient TD(0) for estimating $V_\theta \approx V^\pi$
Input:

- The current policy π
- a differentiable, parameter-dependent function $V_\theta : \mathcal{S} \to \mathbb{R}$
- learning rate α

Initialize: Value function weights $\theta \in \mathbb{R}^d$ arbitrarily

for $k = 1, 2, \dots, K$ episodes:
  Initialize s_0
  for $t = 0, 1, \dots, T-1$:
    Apply action $a_t \sim \pi(\cdot|s_t)$
    Observe r_t and s_{t+1}
    $\theta \leftarrow \theta + \alpha[r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)]\nabla_\theta V_\theta(s_t)$
    if s_{t+1} is **terminal then** STOP

Note: n-step version TD(n)
The above algorithm can be extended towards n-step bootstrapping by replacing the TD(0) target $r_t + \gamma V_\theta(s_{t+1})$ with the n-step bootstrapped return

$$\begin{aligned} g_{t:t+n} =& r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \\ &+ \gamma^{n-1} r_{t+n-1} + \gamma^n V_\theta(s_{t+n}). \end{aligned}$$

The most challenging part in the implementation is the assessment of the remaining number of steps (i.e., when $T - t < n$).

For details, see (Sutton and Barto 1998, Ch. 9.4).

Batch learning

Batch learning / experience replay

- As discussed during the TD learning lecture, training on individual samples can be slow.
- This also applies to SGD-based updates with bootstrapping.
- Alternative: Batch learning methods

Mini-batch semi-gradient TD(0) with experience replay
Based on a dataset $\mathcal{D} = \{(s_0, V^\pi(s_0)), \dots, (s_N, V^\pi(s_N))\}$, repeat

- Sample uniformly b state-value pairs from experience (so-called mini batches), i.e., $\mathcal{B} = \{(s_i, V^\pi(s_i))\}_{i=1}^b \subset \mathcal{D}$:

$$(s_i, V^\pi(s_i)) \sim \mathcal{D}.$$

- Apply (semi) SGD update step:

$$\theta \leftarrow \theta + \frac{\alpha}{b} \sum_{(s_i, V^\pi(s_i)) \in \mathcal{B}} [V^\pi(s_i) - V_\theta(s_i)] \nabla_\theta V_\theta(s_i).$$

Remarks:

- Universally applicable: V_θ can be any differentiable function.

- The true target V^{π} is usually approximated by MC or TD targets.

Least squares policy evaluation

Linear modeling of value functions

- Let's assume that we choose an approximation that is *linear* w.r.t. the weights θ:

$$V_\theta(s) = \theta^\top \psi(s) = \sum_{k=1}^{d} \theta_k \psi_k(s).$$

- What's the gradient in this case? $\Rightarrow$ it's simply the feature vector,

$$\nabla_\theta V_\theta(s) = \nabla_\theta \theta^\top \psi(s) = \psi(s).$$

- Our semi-gradient TD(0) update (3) thus simply becomes

$$\begin{aligned}\theta \leftarrow & \theta + \alpha[r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)]\nabla_\theta V_\theta(s_t) \\ &= \theta + \alpha\left[r_t + \gamma\theta^\top\psi(s_{t+1}) - \theta^\top\psi(s_t)\right]\psi(s_t) \\ &= \theta + \alpha\left[r_t + \gamma\theta^\top\psi_{t+1} - \theta^\top\psi_t\right]\psi_t = \theta + \alpha\left[r_t + \theta^\top(\gamma\psi_{t+1} - \psi_t)\right]\psi_t,\end{aligned}$$

  where we have introduced ψ_t for $\psi(s_t)$.

- Similarly, in **batch mode**, our bootstrapped version of the on-policy prediction objective (1) becomes

$$L(\theta) = \sum_{t=0}^{T-1} \left(r_t + \gamma \theta^\top \psi(s_{t+1}) - \theta^\top \psi(s_t) \right)^2 = \sum_{t=0}^{T} \left(r_t - \left[\psi_t^\top - \gamma \psi_{t+1}^\top \right] \theta \right)^2.$$

Least Squares TD learning (LSTD)

- Collect data:

$$y = \begin{bmatrix} r_0 \\ \vdots \\ r_{T-1} \end{bmatrix}, \qquad \Psi = \begin{bmatrix} \psi_0^\top - \gamma \psi_1^\top \\ \vdots \\ \psi_{T-1}^\top - \gamma \psi_T^\top \end{bmatrix}.$$

- Loss function:

$$L(\theta) = \sum_{t=0}^{T-1} \left(r_t - \left[\psi_t^\top - \gamma \psi_{t+1}^\top \right] \theta \right)^2 = \frac{1}{N} \| y - \Psi \theta \|_F^2.$$

- Optimal solution (a.k.a. **TD fixed point** (Sutton and Barto 1998, Sec. 9.4)):

$$\theta^* = (\Psi^\top \Psi)^{-1} \Psi^\top y = \Psi^\dagger y.$$

- Estimated value function:

$$V_{\theta^*}(s) = {\theta^*}^\top \psi(s) = \sum_{k=1}^{d} \theta_k^* \psi_k(s).$$

Recall: Linear regression

- Model: $\hat{y} = f_\Theta(x) = \Theta^\top x$.
- Loss function $L(\Theta)$ (MSE):

$$\frac{1}{N} \sum_{k=1}^{N} \left\| \Theta^\top x_k - y_k \right\|_2^2 = \frac{1}{N} \| X\Theta - Y \|_F^2.$$

- Batch data:

$$X = \begin{pmatrix} x_1^\top \\ \vdots \\ x_N^\top \end{pmatrix}, \quad Y = \begin{pmatrix} y_1^\top \\ \vdots \\ y_N^\top \end{pmatrix}$$

- Optimal solution (Θ^*):

$$\Theta^* = (X^\top X)^{-1} X^\top Y = X^\dagger Y.$$

Summary / what you have learned

Summary / what you have learned

- To cover large or continuous state spaces **function approximation** is required.
- **On-policy prediction** is straightforward with function approximation:
  - Transfer of incremental learning from the tabular case to **gradient descent for** θ.
  - **Stochastic gradient descent** allows step-by-step based updates.
- Gradient-based prediction is **not risk free** (especially non-linear case):
  - challenging optimization problems (descent directions, learning rates, etc.),
  - local optima vs. global optimum.
- If **bootstrapping** is applied, the update target depends on θ.
  - True gradient becomes computationally more complex.
  - **Semi-gradient** methods reduce computational burden at accuracy costs.
- **Batch learning** is often more efficient for training.

- **Linear models** can be trained in a single step using the pseudo-inverse.

References

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.